



Fundamentals of Security Technology

Contents

Introduction	3
Overview of the Ext4 File System	3
Importance of Understanding Ext4 File System for Security Technology	3
Purpose and Objectives of the Assessment.....	4
Partition Name and Block Size Management	4
Importance of Partition Naming and Block Size	4
Implementation Process: Challenges and Solutions.....	4
Documentation and Analysis of Partition Name and Block Size.....	5
Ext4 Fundamental Structures	5
Overview of Main Ext4 Data Structures	5
Practical Observation: Connecting Theoretical Concepts	6
Analyzing GPT to Superblock Connections	6
Calculation of Total Disk Space:	9
Superblock to Root Folder Relations	9
Calculation of Free Space:	10
Root Folder to File Inode Connections	10
Root Folder to File Inode Connections:	11
File Inode to File Contents Pathway	11
Documenting File Location in Ext4	12
Tracing File Location: Practical Demonstration	12
GPT to Superblock Mapping.....	12
Analysis Before File Deletion	13
Superblock to Root Folder Navigation.....	13
GPT to Superblock Mapping	14
Root Folder to File Inode Identification.....	14
File Inode to File Contents Examination.....	15
Conclusion of Analysis Before File Deletion	17
Effect of File Deletion on Ext4.....	18
Understanding File Deletion Mechanics:.....	18

Impact on File System Structure:.....	18
Changes in Data Structures Post File Deletion:	18
Implications of File Deletion:	19
Conclusion:	19
Contents of a Deleted File and Its Impact.....	19
Analysis of Deleted File Contents:	19
Privacy Implications and Consequences Post Deletion:	20
Challenges in Ext4 Forensics	20
Personal Experience and Mitigation:.....	21
Factors Affecting Forensic Analysis:	22
Limitations and Challenges Faced by Analysts:	22
Reflective Conclusion.....	23
References	24

Table of Figures

Figure 1: Ext2/Ext3/Ext4 File System Format	3
Figure 2: Successful Partition Created	7
Figure 3: Disk and its Partitions	8
Figure 4:Overall File System Structure.....	9
Figure 5: Details of superblock and Inode	10
Figure 6: File creation Commands	13
Figure 7: Successful File Creation	13
Figure 8: Hex view of Data inside file.....	13
Figure 9: txt file presence in HxD	14
Figure 10: File Location in Hex	15
Figure 11:File Inode to File Contents	16
Figure 12: Other Metadata of the disk	17

REPORT ON EXT4 FILE SYSTEM ANALYSIS

Introduction

The Ext4 File System is a crucial stronghold in modern computing. It describes precisely how data is arranged and stored on computer hard discs. It is indispensable not only in the world of security technology, but once you understand its intricacies, it becomes necessary to stop cyber terrorism and protect digital data. This research examines the Ext4 File System. It will examine its structure, function and the consequences of playing with file systems--a particularly dangerous enterprise for a forensic specialist.

Overview of the Ext4 File System

Inside the Linux family, Ext4 is an extension of its predecessors Ext2 and Ext3. This is a major breakthrough in data storage and retrieval. Because of Ext4's scalability, performance and reliability many Linux distributions come pre-installed with it. Several data structures, such as the superblock, inode, and data blocks, make up its architecture. These all work together to decide file storage and retrieval [1].

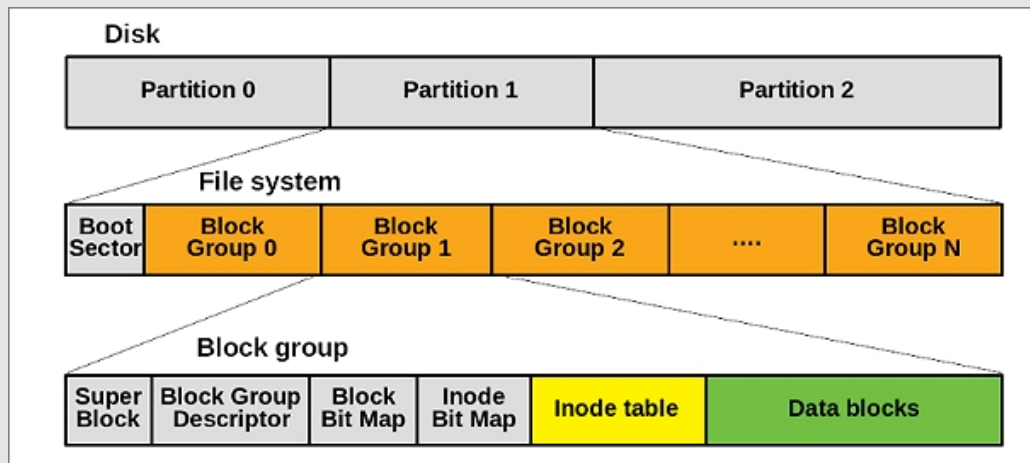


Figure 1: Ext2/Ext3/Ext4 File System Format

Importance of Understanding Ext4 File System for Security Technology

In Security Technology, a deep understanding of Ext4 is necessary. In light of today's digital threats, a systematic knowledge about the file system is needed to detect weaknesses and imagine potential dangers so as to develop solid security mechanism. Because malicious actors always take advantage of any loopholes they can find in file systems, to protect data integrity and confidentiality we must have an intimate knowledge of Ext4.

Purpose and Objectives of the Assessment

Through implementation and exploration, this assessment aims to understand the finer points of Ext4 File System. In short, it is an attempt to penetrate the inner workings of these seemingly mysterious mechanisms which determine how we store and delete files--and by what means they are consequently stored in extinct or fossilized forms within computers. Through this, the assessment endeavors to achieve the following objectives:

1. **Examination of Ext4 Structure:** Examining and understanding the core data structures within Ext4 in order to appreciate their interrelationships, and functionalities.
2. **Traceability of File Operations:** Explaining the hierarchical connections between critical Ext4 structures like GPT (GUID Partition Table) and file contents.
3. **Understanding Forensic Challenges:** Facing obstacles: Recovering deleted files, measuring the effects of deletion on file system integrity or—at even a more basic level--detecting potential privacy concerns.

In the subsequent chapters this report will scrutinize each element of Ext4 in turn, reflecting upon our own experiences and drawing on theoretical study.

Partition Name and Block Size Management

Importance of Partition Naming and Block Size

Deciding on partition names and block sizes are important for setting up, as well as fine-tuning a file system. A partition's name is a kind of sign. By just glancing at it, you can know where to put which storage object inside. In addition, block size has a direct impact on the efficiency in which files are stored. It can affect things like disc space used and file access speed.

Standing on the actual battlefield for this example 8GB storage medium, in real operation steps one involves creating a Ext4 partition for it. Selecting an appropriate block size and naming convention involves trade-offs in performance, storage efficiency or portability. For large files, a larger block size will speed up read-write. However, for small files it wastes space. By contrast, a relatively smaller block size saves space but might add overhead for larger files.

Implementation Process: Challenges and Solutions

Challenges may arise in implementation, especially because of deciding upon an optimal block size. For example, setting specific block size sometimes produces compatibility problems that result in errors or obstacles during the partitioning process. The key is to take iterative steps, starting with larger block sizes and gradually shrinking until compatible.

Part of this assessment includes the recording of these challenges and subsequent solutions. It not only demonstrates the application of theoretical knowledge in practice but also analyzes how administrative obstacles to implementation are overcome. In addition to describing the methodology used, this documentation is also a guarantee of credibility for the report itself by proving just how much insight was given into partitioning Ext4. [2]

Documentation and Analysis of Partition Name and Block Size

A detailed record must be made of the partition name chosen and its block size after creating a successful partition. Documentation should reflect the reasons for these choices, in a way that makes us feel like we understood why file systems were decided on pursuing this path.

The examination of how choosing the block size and partition name affects Ext4's structure is crucial to this section. This includes a thorough examination of the impact these factors have on system performance, file storage, and retrieval. Finding patterns in this study helps establish links between the parameters selected and the functionality of the Ext4 File System.

The rest of the report's chapters will look at file locations and Ext4 core structures after this introduction. Next, we will examine the consequences of file deletion and discuss various issues that arose during the Ext4ji forensics examination.

Ext4 Fundamental Structures

Overview of Main Ext4 Data Structures

The Ext4 File System requires key data structures in order to allocate, store, and retrieve files. Comprehending these structures is essential to grasping the fundamental mechanisms that regulate Ext4 functions:

1. **Superblock:** A base construction, the Superblock is. It includes the most fundamental file system metadata, including block size, inode count, and predicted total block count. It is crucial to mounting operations for both starting and checking the file system.
2. **Inode Table:** Inodes, which stand for index nodes, are vital data structures that store information about files and directories. An inode stores metadata on each file in Ext4, including permissions, timestamps and pointers to data blocks [3].
3. **Block Bitmap and Inode Bitmap:** The file system uses bitmaps to record the allotment status of data blocks and inodes. They show which blocks or inodes are being used and which can be allocated.
4. **Data Blocks:** These are real storage tanks holding file contents. To manage file storage efficiently, Ext4 uses different types of blocks. These include data blocks (which store the actual content), indirect blocks and extent blocks.

Practical Observation: Connecting Theoretical Concepts

In terms of the practical aspects of Ext4 file system analysis, there is a very close connection between theoretical knowledge and implementation. This practical observation is thus a very significant link between academic understanding of the structures in Ext4, and actual survival under conditions forced upon it.

Ext4 Fundamental Structures Exploration

This provided a theoretical basis for using different commands to look at the underpinnings of Ext4 such as: `dumpe2fs`, shedding light on essential elements like the superblock and inode structure or block allocation. These tools helped extract vital data about the layout of files in this old file system, and allowed me to organize documents till I'd worked out an elegant pattern for pathways through superblocks down to individual inodes that created a complex web.

Finding the Location of a File & Deleting It in Ext4

The actual implementation required creating a file, making it and then deleting the file out of Ext 4. Making a file, the pathways from GPT through intermediate Ext4 data structures are greeted by play-minded eyes for its final location or vacation spot down there on an LVM extended volume even in California would be considered reasonable. In addition, by looking into the impact of file deletion we can see changes in system structure and understand privacy considerations after a delete.

Challenges Encountered and Recovery Attempts

However, issues arose when they attempted to recover erased files. Issues with partition dismounting and identifying running processes that are interfering with un-mounting caused issues with running commands such as `extundelete`. These challenges emphasized the need of following the right protocols for demounting and the necessity of maintaining stability at all costs. Furthermore, insufficient dismounting might cause issues with recovery efforts.

Insights into Theoretical Application

The experiment demonstrated the practical application of these theoretical ideas in real-world scenarios, highlighting the complexity of forensic analysis. Even more, actual use allowed for the realization of how complex and interconnected digital forensics were, in addition to theoretical understanding illuminating the structures of Ext4.

Analyzing GPT to Superblock Connections

The practical analysis of the Ext4 file system began with partitioning a drive (`/dev/sdc1`) and formatting it as an Ext4 filesystem. The following commands were executed:

- lsblk
- sudo mkfs.ext4 -b 4096 -L ArshadAshraf /dev/sdc1
- sudo dumpe2fs /dev/sdc1 | grep "superblock"

```
[x]~[parrot@parrot-vm]~[~]
$ sudo mkfs.ext4 -b 4096 -L ArshadAshraf /dev/sdc1
mke2fs 1.46.6 (1-Feb-2023)
/dev/sdc1 contains a vfat file system labelled 'ARSHADASHRA'
Proceed anyway? (y,N) y
Creating filesystem with 47104 4k blocks and 47104 inodes
Filesystem UUID: b64e02c3-b18c-4bc2-8d43-6ba458096d88
Superblock backups stored on blocks:
    32768

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done
```

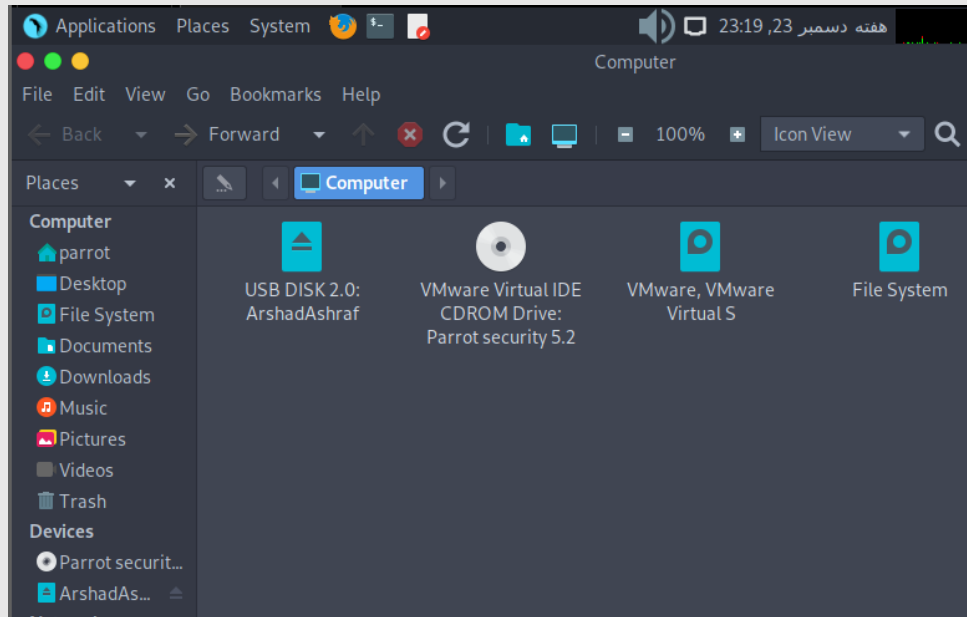


Figure 2: Successful Partition Created

The output of these commands gave an inkling as to how the GPT (GUID Partition Table) linked up with Superblocks, and hinted at where primary and backup superblock locations could be found. The superblocks give the file system its parameter and characteristics.

Looking at the relationship between GUID Partition Table (GPT) and Superblock in Ext4 reveals significant information about what lies beneath as regards overall structure of storage, data organization. On modern disks, the GPT is a partitioning scheme defining partitions and their boundaries. However, the Superblock is an essential metadata structure within each Ext4 file system partition. It contains important information about the internal layout and configuration of a given file system.

Upon inspecting the GPT to Superblock connections in the provided practical exploration, we obtained insightful data:

lsblk Output:

- The lsblk command displays the disk and its partitions. In this instance, /dev/sdc1 is the 7.5GB partition created for the Ext4 file system.


```
[parrot@parrot-vm]~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda          8:0    0   20G  0 disk
└─sda1       8:1    0   20G  0 part /home
sdc          8:32   1    7.5G  0 disk
└─sdc1       8:33   1   184M  0 part
sr0         11:0    1    4.8G  0 rom
```

Figure 3: Disk and its Partitions

Superblock Metadata:

- **Primary Superblock:** Located at block 0.
- **Backup Superblock:** Positioned at block 32768.
- **Group Descriptors:** Associated with the primary and backup superblocks.

Inode-Related Details:

- **Inode Count:** Total of 47104 inodes available.
- **Inodes per Group:** Configuration with 23552 inodes per group.
- **Inode Blocks per Group:** Set to 1472 inode blocks per group.
- **Inode Size:** Each inode size specified as 256 bytes.

```

[parrot@parrot-vm]~$ sudo dumpe2fs /dev/sdc1
dumpe2fs 1.46.6 (1-Feb-2023)
Filesystem volume name:   ArshadAshraf
Last mounted on:         <not available>
Filesystem UUID:         b64e02c3-b18c-4bc2-8d43-6ba458096d8
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resize_inode dir_index
                           extent 64bit flex_bg sparse_super large_file huge_file dir_nlink
Filesystem flags:         signed_directory_hash
Default mount options:    user_xattr acl
Filesystem state:         clean
Errors behavior:          Continue
Filesystem OS type:       Linux
Inode count:              47104
Block count:              47104
Reserved block count:    2355
Overhead clusters:       7092
Free blocks:              40006
Free inodes:              47093
First block:              0
Block size:               4096
Fragment size:            4096
Group descriptor size:    64
Reserved GDT blocks:     22
Blocks per group:         32768
Fragments per group:     32768
Inodes per group:         23552
Inode blocks per group:   1472
Flex block group size:    16
Filesystem created:       Sat Dec 23 23:14:25 2023

Directory Hash Seed:      813c6342-ac41-4c5f-9c5f-a7eep3225
Journal backup:           inode blocks
Checksum type:            crc32c
Checksum:                 0xfa162267
Journal features:         journal_64bit journal_checksum_v3
Total journal size:       16M
Total journal blocks:     4096
Max transaction length:   4096
Fast commit length:       0
Journal sequence:         0x00000002
Journal start:            1
Journal checksum type:    crc32c
Journal checksum:         0x9707cb65

Group 0: (Blocks 0-32767) csum 0x6334 [ITABLE_ZEROED]
  Primary superblock at 0, Group descriptors at 1-1
  Reserved GDT blocks at 2-23
  Block bitmap at 24 (+24), csum 0x3ef38b48
  Inode bitmap at 26 (+26), csum 0xa2495445
  Inode table at 28-1499 (+28)
  25694 free blocks, 23541 free inodes, 2 directories, 2354
  Free blocks: 7074-32767
  Free inodes: 12-23552
Group 1: (Blocks 32768-47103) csum 0x836e [INODE_UNINIT, IT
  Backup superblock at 32768, Group descriptors at 32769-32
  Reserved GDT blocks at 32770-32791
  Block bitmap at 25 (bg #0 + 25), csum 0x3d53aade
  Inode bitmap at 27 (bg #0 + 27), csum 0x00000000
  Inode table at 1500-2971 (bg #0 + 1500)
  14312 free blocks, 23552 free inodes, 0 directories, 2355
  Free blocks: 32792-47103
  Free inodes: 23553-47104

```

Figure 4:Overall File System Structure

Calculation of Total Disk Space:

Total Disk Space = Block Size × Total Blocks

Total Disk Space = 4096 bytes/block × 47104 blocks

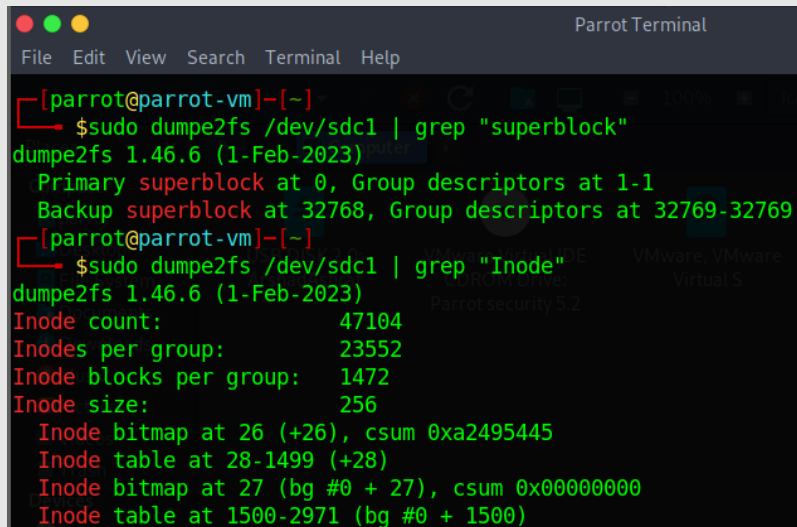
Total Disk Space ≈ 193,294,784 bytes ≈ 184 MB (approx.)

Superblock to Root Folder Relations

Analyzing the relationship between the Superblock and Root Folder in an Ext4 file system shows something of its structural hierarchy and organization. As a critical metadata structure, the Superblock contains important data about file system layout. The root folder location and its attributes are in there too [4]. Concerning the superblocks, meanwhile, this was accompanied by an investigation into inode-related information. The commands used for this purpose were:

- `sudo dumpe2fs /dev/sdc1 | grep "superblock"`
- `sudo dumpe2fs /dev/sdc1 | grep "inode"`
- `sudo dumpe2fs /dev/sdc1 # Explore Overall File System Structure`

These commands provide information about the Inode count, the number of Inodes per group, and other important statistics. The superblock-to-root folder relations are basic in unravelling the hierarchical structure of the file system.



```

[parrot@parrot-vm]-[~]
$ sudo dumpe2fs /dev/sdc1 | grep "superblock"
dumpe2fs 1.46.6 (1-Feb-2023)
  Primary superblock at 0, Group descriptors at 1-1
  Backup superblock at 32768, Group descriptors at 32769-32769

[parrot@parrot-vm]-[~]
$ sudo dumpe2fs /dev/sdc1 | grep "Inode"
dumpe2fs 1.46.6 (1-Feb-2023)
Inode count:                47104
Inodes per group:           23552
Inode blocks per group:     1472
Inode size:                  256
  Inode bitmap at 26 (+26), csum 0xa2495445
  Inode table at 28-1499 (+28)
  Inode bitmap at 27 (bg #0 + 27), csum 0x00000000
  Inode table at 1500-2971 (bg #0 + 1500)

```

Figure 5: Details of superblock and Inode

Calculation of Free Space:

The **dumpe2fs** command provides details about the free blocks and free inodes:

- Free blocks: 40006 blocks
- Free inodes: 47093 inodes

Free Space = Free Blocks × Block Size

Free Space = 40006 blocks × 4096 bytes/block

Free Space ≈ 163,954,176 bytes ≈ 156 MB (approx.)

Therefore, from a total disk space of approximately 184 MB, there's around 156 MB of free space available in the "ArshadAshraf" partition.

Root Folder to File Inode Connections

The relationship between the root folder inode and file inodes is very important to understanding organization and access of a file system. The root directory in an Ext4 file system is the starting point for all directories represented by an inode (typically inode 2).

Root Folder Information:

- **Root Inode Number:** Typically, inode 2.
- **Directory Entries:** Contains references to various files and subdirectories.

Root Folder to File Inode Connections:

The root folder is actually just a directory holding references to file and subdirectory entries within the underlying filesystem. Each entry in the root directory is matched to an inode, representing a file or subdirectory. Understanding the Connection:

1. **Root Inode (inode 2):** The root directory has its own inum, usually inode number 2. The inode in this directory is metadata about the latter itself--permissions, owner information, timestamps and other attributes.
2. **Directory Entries:** The entries within the root directory refer to file inodes. Each entry is a filename with an attached inode number. These entries create an immediate relationship between the root directory and its associated inodes, allowing files to be read out.

A new entry is added to the root directory's contents when a file is created within it. This entry points to an inode, which uniquely identifies the file. The inode, which stores metadata about the file (permissions, ownership information, time stamps), is most importantly an index of data block pointers for the actual file contents.

Linkages between the root folder and file inodes are created through directory entries linking to their own respective inodes. These inodes contain important information about the file, allowing the operating system to find and bring back its data blocks. The inode provides access to the metadata about a file and its data blocks. It is an index for where on disk the physical extent of a given file can be found [5].

File Inode to File Contents Pathway

The steps from the inode of a file to its actual contents are elucidated in detail throughout this process, which shows how data associated with one particular file is found by an Ext4-based system.

Understanding the Pathway:

1. **Inode Metadata:** Each file in a Ext4 filesystem has an associated inode containing metadata such as permissions, timestamps, ownership details and crucially pointers to the data blocks on disk storing that particular file's content.
2. **Data Block Pointers:** Inside the inode, pointers point to data blocks inside which are stored the contents of files. These pointers are critical in locating the data blocks that really contain a file's information.
3. **Data Blocks:** On the storage device, actual file contents are divided into data blocks. Here, the pointers in the inode guide file system through to these particular blocks where content sits [6].

Results and Analysis:

1. **Inode Information:** From the provided data:
 - **Inode Count:** 47104 inodes are available in the file system.

- **Inode Size:** 256 bytes for each inode.
 - **Inode Table and Bitmap:** Locations and details about inode structures and allocation.
2. **Block Pointers:** Each inode contains pointers to data blocks. These pointers guide the file system to the actual blocks storing the file's content.

Pathway and Functionality:

1. **File Access:** When a file has to be read or edited, the inode for that file is looked up by the file system.
2. **Pointer Navigation:** These pointers inside the inode then point to particular data blocks containing the file's content. These pointers may directly point to data blocks or indirect, block-pointing ones.
3. **Data Retrieval:** Based on these suggestions, the file system extracts data blocks containing the contents of that particular file. After that, the data blocks are read or altered as necessary.
4. **Efficiency and Optimization:** To achieve efficient data retrieval, Ext4 optimizes block allocation and uses different types of pointers (direct, indirect, doubly indirect) to find the location where a file's content can be quickly located.

Knowing this pathway is the condition for data retrieval and manipulation within Ext4, so that you can go rapidly from any inode to its associated data blocks.

Documenting File Location in Ext4

This involves a series of commands detailed in the following illustrating how files are located within Ext4, from GPT to the actual file data. The purpose of this practical demonstration is to show the complex path from GUID Partition Table (GPT) down to actual contents of a file.

Tracing File Location: Practical Demonstration

GPT to Superblock Mapping

First, you need to make a directory 'ArshadAshraf' in the '/media', and within this create a text file named ArshadAashraf.txt containing just the line of text above reading "Arshad". The following commands demonstrate this:

- `cd /media/parrot/ArshadAshraf`
- `sudo touch ArshadAshraf.txt`
- `echo "Arshad" | sudo tee ArshadAshraf.txt`

```
[x]-[parrot@parrot-vm]-[~]
$cd /media/parrot/ArshadAshraf
[parrot@parrot-vm]-[/media/parrot/ArshadAshraf]
$sudo touch ArshadAshraf.txt
[parrot@parrot-vm]-[/media/parrot/ArshadAshraf]
$echo "Arshad" | sudo tee ArshadAshraf.txt
Arshad
[parrot@parrot-vm]-[/media/parrot/ArshadAshraf]
$
```

Figure 6: File creation Commands

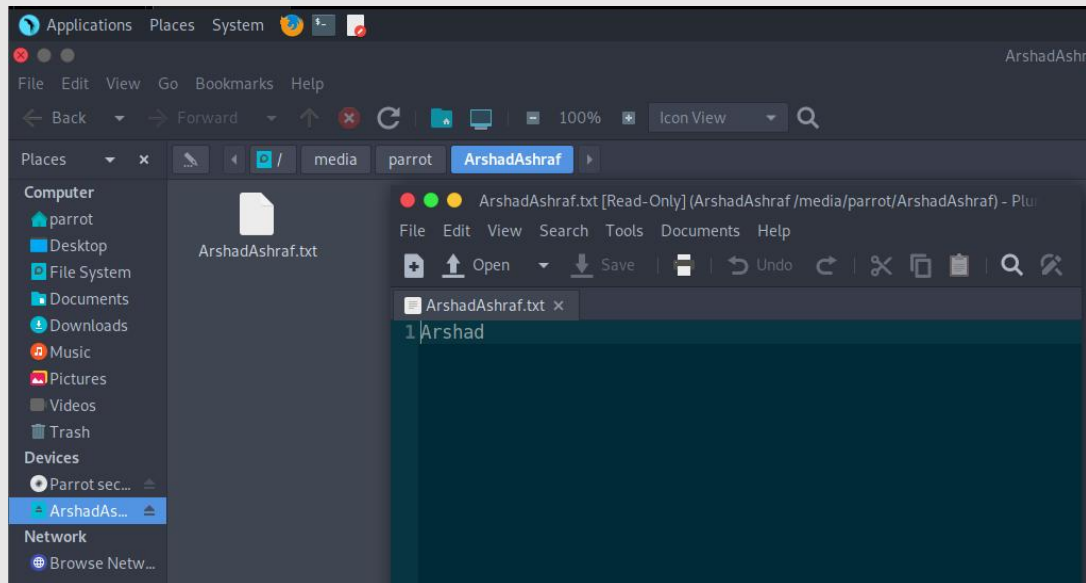


Figure 7: Successful File Creation

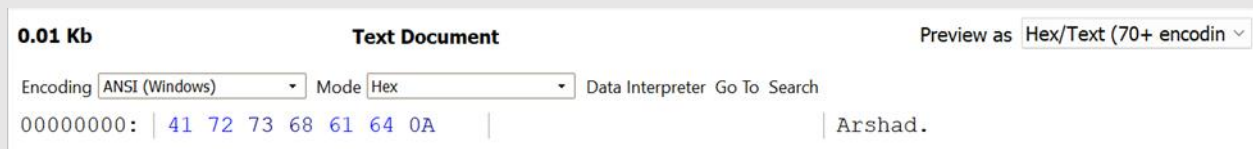


Figure 8: Hex view of Data inside file

Analysis Before File Deletion

Finding the file before it is deleted – To get information on where the file resides and so forth, there are still several commands involved. Here's an outline of the commands used and analysis:

Superblock to Root Folder Navigation

```
sudo ls -l /media/parrot/ArshadAshraf
```

```
[parrot@parrot-vm]-[/media/parrot/ArshadAshraf]
$sudo ls -l
[sudo] password for parrot:
total 20
-rw-r--r-- 1 root root 7 02:35 24 دسمبر ArshadAshraf.txt
drwx----- 2 root root 16384 23:14 23 دسمبر lost+found
[parrot@parrot-vm]-[/media/parrot/ArshadAshraf]
$
```

This command lists the files in the 'ArshadAshraf' directory, showing the file size, permissions, and timestamp, providing an overview of the file system structure.

00000250:	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000260:	00 01 01 04 00 00 00 2E	00 74 00 78 00 74 00 01	t.x.t..

Figure 9: txt file presence in HxD

`sudo ls -l /media/parrot/ArshadAshraf` This command gives a detailed list of the files in 'ArshadAshraf', including file sizes, permissions and timestamps. Nevertheless, in this instance the 'ArshadAshraf.txt' is shown as 7 bytes long (a small file). These rights are read and write for root; read-only for others (or access restricted). The timestamp indicates that the file was last modified on December 24 at 02: 35. The fact that there is a 'lost+found' directory, created with restricted permissions means this must be the place files end up if your system fails.

GPT to Superblock Mapping

<code>sudo debugfs -R "blocks /ArshadAshraf.txt" /dev/sdc1</code>	<code>[parrot@parrot-vm]~/media/parrot/ArshadAshraf \$sudo debugfs -R "blocks /ArshadAshraf.txt" /dev/sdc1 debugfs 1.46.6 (1-Feb-2023) 7074</code>
---	--

Using `debugfs`, we can map the file 'ArshadAshraf.txt' to individual blocks within the file system and pick out where its contents are physically stored on disk.

The command `sudo debugfs -R "blocks /ArshadAshraf.txt" /dev/sdc1` compiled lines from the 'debugfs' directory gives important block-level details about file ArshadAshraf. txt in particular Output '7074' is actually the address of a particular block within that Ext4 file system, in which this file lives. It's important to understand block allocation because it allows you to track where the content of a file is physically located on the disk. This correspondence between a file and the list of blocks to which it is associated makes possible an even more detailed investigation into the internal structure of the file system, as for example in linking up this particular inode with its belonging folder one level higher Bottom.

Root Folder to File Inode Identification

<code>sudo ls -li /media/parrot/ArshadAshraf/ArshadAshraf.txt</code>	<code>[parrot@parrot-vm]~/media/parrot/ArshadAshraf \$sudo ls -li /media/parrot/ArshadAshraf/ArshadAshraf.txt 12 /media/parrot/ArshadAshraf/ArshadAshraf.txt</code>
--	---

This command, showing as the file inode number in 'ArshadAshraf', is a unique identifier for files under the file system.

When the command `sudo ls -li` is issued and produces this as its output, then file inode number 12 is returned for 'ArshadAshraf.txt' in directory ArshadAshraf The inode is a unique identifier assigned to each file in the system. This command, by linking the file name to its inode number within the directory structure, makes it easier for users to track files. Inode numbers form the interface between a file's metadata and its actual data blocks on the disk; they play an important part in referencing files.

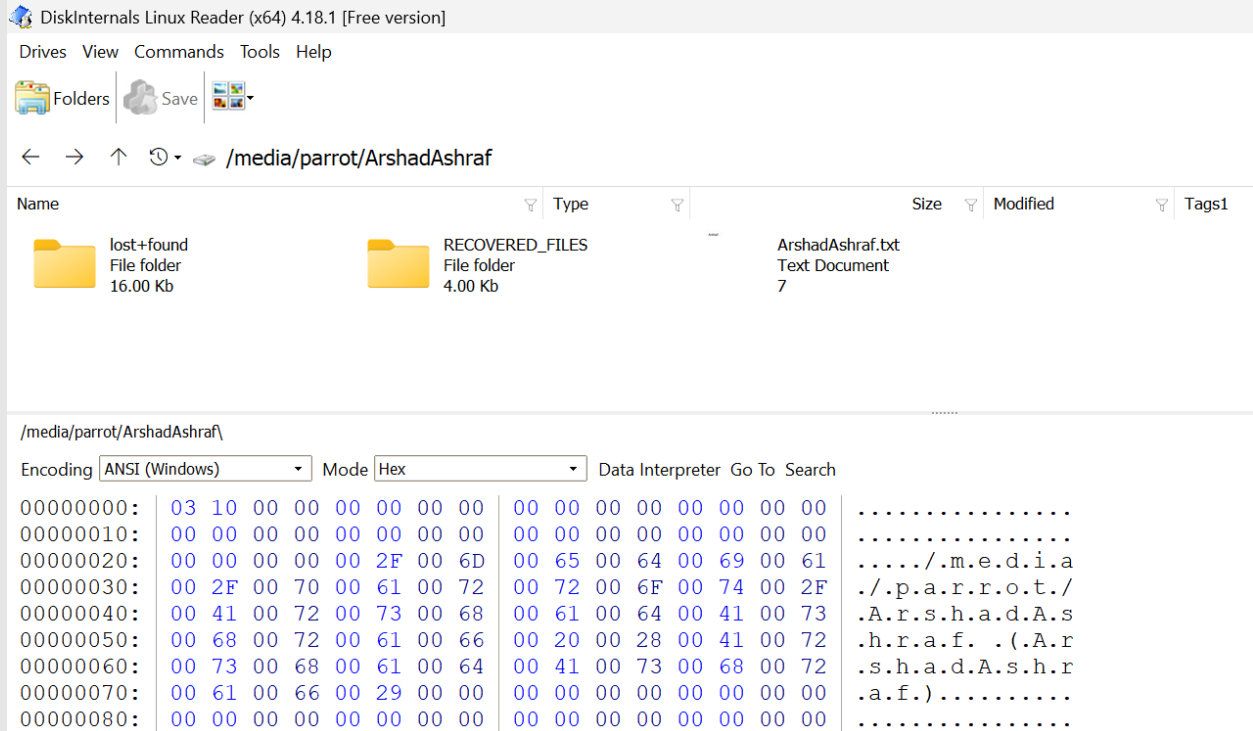


Figure 10: File Location in Hex

Text	Hex
m	6D
E	65
D	64
I	69
a	61
/	2F
p	70
r	72
o	6f
t	74
h	68
f	66
()	28 29

File Inode to File Contents Examination

```
sudo debugfs -R "stat /ArshadAshraf.txt" /dev/sdc1
```

By using **debugfs** and the 'stat' command, details about the file, including size, inode, permissions, and timestamps, can be obtained. This aids in understanding the file's inode-level information.


```
Inode: 12   Type: regular   Mode: 0644   Flags: 0x80000
Generation: 3001194338   Version: 0x00000000:00000002
User:      0   Group:      0   Project:      0   Size: 7
File ACL: 0
Links: 1   Blockcount: 8
Fragment: Address: 0   Number: 0   Size: 0
  ctime: 0x658752a5:a6e7d238 -- Sun Dec 24 02:35:33 2023
  atime: 0x65875286:9e29dbc4 -- Sun Dec 24 02:35:02 2023
  mtime: 0x658752a5:a6e7d238 -- Sun Dec 24 02:35:33 2023
  crtime: 0x65875286:9e29dbc4 -- Sun Dec 24 02:35:02 2023
Size of extra inode fields: 32
Inode checksum: 0x6b84bf78
EXTENTS:
(0):7074
(END)
```

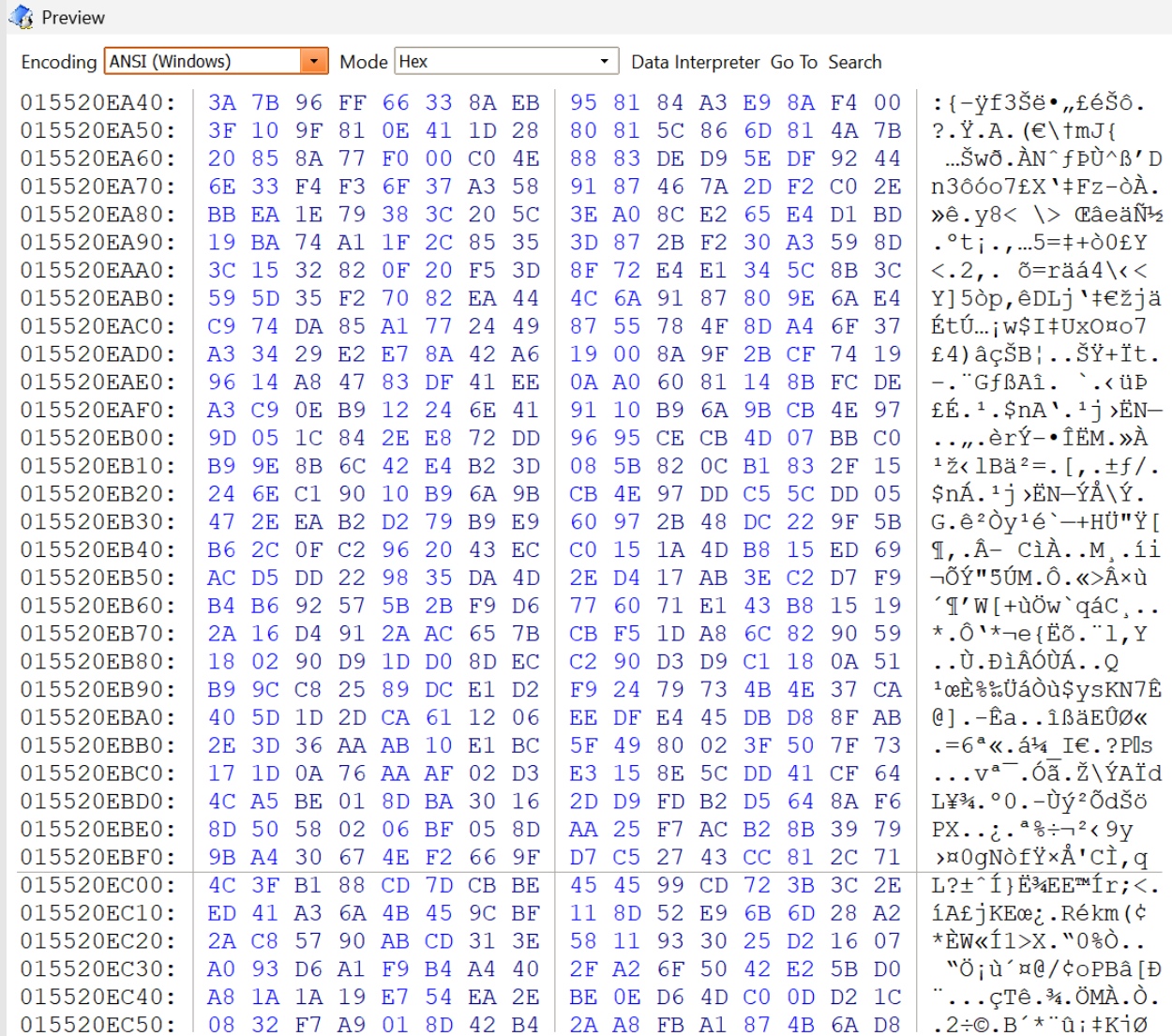
"USB DISK 2.0: ArshadAshraf" selected

Figure 11:File Inode to File Contents

Here's an analysis of the output:

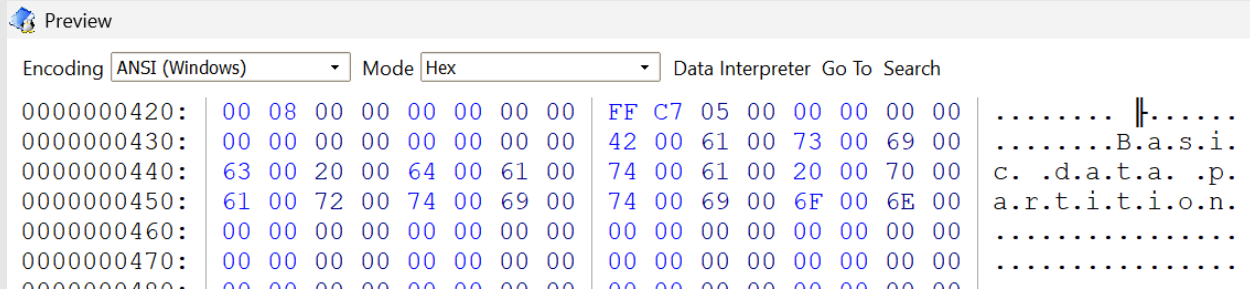
- **Inode Information:** Inode number '12' is associated with a regular file having permissions set to '0644'. This file has a size of '7' bytes and a block count of '8'.
- **Timestamps:** The file shows several timestamps:
 - **Creation time (ctime):** Dec 24, 2023, 02:35:02
 - **Last access time (atime):** Dec 24, 2023, 02:35:02
 - **Last modification time (mtime):** Dec 24, 2023, 02:35:33
 - **Change time (ctime):** Dec 24, 2023, 02:35:33
- **EXTENTS:** The file 'ArshadAshraf.txt' is composed of blocks of data, and the extent display indicates where in a given article the contents fall. In the output it is a block address '7074' where lies the file content.

With this information, you can get a complete picture of the structure and contents of your file.



Conclusion of Analysis Before File Deletion

The analyzer presented an all-around picture of this file 'ArshadAshraf.txt' in the Ext4 file system: its location, size, permissions and inode attributes are all part of the show here. This practical demonstration illuminates the complex route from GPT through to structures of the file system down to individual files themselves, so that we can see how each structural element is connected throughout.



Effect of File Deletion on Ext4

Deleting a file in the Ext4 File System involves changes to its structure which affect how information is stored and managed. Here's an in-depth exploration of the mechanics and implications of file deletion on Ext4:

- `sudo rm /media/parrot/ArshadAshraf/ArshadAshraf.txt`
- `ls -l /media/parrot/ArshadAshraf`

```
[x]~[parrot@parrot-vm]~[ /media/parrot/ArshadAshraf ]
$ sudo rm /media/parrot/ArshadAshraf/ArshadAshraf.txt
[parrot@parrot-vm]~[ /media/parrot/ArshadAshraf ]
$ ls -l /media/parrot/ArshadAshraf
total 16
drwx----- 2 root root 16384 23:14 23 دسمبر lost+found
```

Understanding File Deletion Mechanics:

A deleted file in Ext4 is not removed immediately from the disk. However, the file's metadata including its inode (a data structure containing information about the file) is marked as free. The occupied space is freed as a result of this process, which makes the file unavailable for use and overwritable. But until such time as the data has been wiped out, it often can still be recovered with specialized recovery tools [7].

Impact on File System Structure:

File deletion has a nuanced impact on the file system structure:

- **Inode Deletion:** In this way, the association between its name and data blocks is dissolved through marking inode of file as free. The inode's metadata is freed, allowing it to be used for new files.
- **Block De-allocation:** Given that these data blocks hold the file's contents, so they are considered free for storing new data. The blocks labelled free space in the block allocation map are these.

Changes in Data Structures Post File Deletion:

Inode Table:

- **Free Inodes:** The number of available inodes increases after file deletion. The inode bitmap marks previously allocated inodes as available for reuse.

Block Allocation Map:

- **Free Blocks:** The blocks previously occupied by the deleted file are marked as available in the block bitmap, enabling them for reuse by subsequent files.

File System Metadata:

- **File System Journal:** Deletion might alter the journal entries, updating the metadata regarding the file system changes.

Implications of File Deletion:

- **Performance Improvement:** Recovering space is capable of improving system performance, because it frees up storage for new files without necessarily extending the file system.
- **Data Security:** Although the file itself may appear to have been emptied, until it is overwritten its contents can still be recovered. For sensitive data, secure methods or tools should be used to prevent recovery.
- **Metadata Changes:** Information about the file's existence and properties are removed, causing a change in directory structure as well records of the file system itself.

Conclusion:

Deleting a file in Ext4 can cause quite some changes to the structure of the allocation portion of the file system. For one thing, it releases both inodes and blocks that were used by this now-gone file. In data recovery scenarios, where deleted files need to be retrieved, understanding of these mechanics is very important. Therefore, you must also realize that the actual contents of the file still might reside on the disk until overwritten. For data protection and security purposes therefore deletes need to be safe ones.

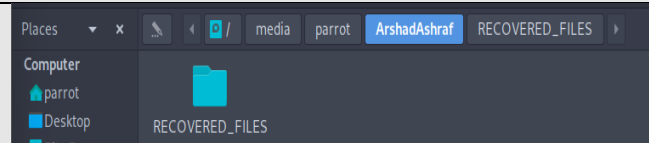
Contents of a Deleted File and Its Impact

Knowing what is in a deleted file, or the consequences of its deletion require knowing how data storage systems work and where the privacy problems arise even after you erase something.

Analysis of Deleted File Contents:

When you delete a file, its contents don't go disappearing from the storage medium. The space occupied by the file is flagged as available for future use by flags in the operating system. This means that until new data fills up the space occupied by a deleted file, its contents can still be recovered with specialized recovery software. You can use the extundelete tool to recover deleted files from an ext3 or ext4 file system. The following steps outline the process of attempting to restore the deleted file 'ArshadAshraf.txt' and analyzing its contents:

- `sudo extundelete --restore-file /media/parrot/ArshadAshraf/ArshadAshraf.txt /dev/sdc1`
- `ls /recovered_files`



This command attempts to retrieve the file 'ArshadAshraf.txt' from drive '/dev/sdc1'. It examines if any recoverable data can be found from the provided file path. After running the recovery command, to inspect the recovered files by checking the directory where extundelete restores the recovered files:

Privacy Implications and Consequences Post Deletion:

1. **Residual Data Concerns:** Even after a file has been deleted, data residues may remain in the disk until overwritten by other information. Tools for recovering such fragments as extundelete can sometimes recoup these hither and yon detritus, raising privacy intrusions if sensitive data lies within the deleted file.
2. **Risks of Data Exposure:** On the other hand, there is a chance that sensitive information included in the recovered files will be exposed on shared or public systems. If information is made public, there may be issues with authorization or data loss.
3. **Potential Mitigation:** Complete data erasure via safe deletion methods or disc wiping is essential to avert such mishaps. Recovery is all but impossible when random data is used to overwrite disc areas occupied by deleted files.
4. **Best Practices for Privacy:** We can encrypt sensitive data, implement secure delete procedures, and be mindful of file access permission issues as some preventive measures against data exposure after deletion.

Understanding the need of restoring erased files informs our understanding of safe deletion practices and data security. Additionally, it helps users avoid potential hazards brought on by leftover data by reminding them to exercise caution while handling sensitive information and to learn the best practices for securely erasing files.

Challenges in Ext4 Forensics

For file recovery though, searching Ext4 systems can present several challenges. Here's a dive into the hurdles and complexities encountered during Ext4 forensics:

1. **File Fragmentation:** Ext4 divides up files by extent-based allocation. It becomes quite difficult to actually recover a broken file. It is necessary to have a basic understanding of how Ext4 distributes files across the disc.
2. **Journaling System:** But the journaling system used in Ext4 makes forensics harder. Recovery might be hampered by the journaling mechanism which overwrites data.

3. **Metadata Complexity:** Problems arise from the complex metadata structures for inode or block mapping. These structures are the key to correct recovery of files, and require experts versed in Ext4 internals.
4. **File Deletion Mechanisms:** In ext4, file deletion doesn't always completely wipe out data right away. To the contrary, by leaving this portion of space blank human judgment is nearly powerless when you try once again in later years to read data from your drive.
5. **Data Overwriting and Fragment Mixing:** Newly saved data can overwrite previously erased content, making recovery even more difficult. Furthermore, data shredding will still place active and deleted data in the same blocks.
6. **Limited Tools and Support:** A number of tools, such as extundelete and debugfs get files back out there but the results are a little up in the air. Since there are yet complete tools for in-depth analysis of many file systems, therefore the depth to which forensic investigation can penetrate is limited.
7. **Time Sensitivity:** Speed is of the essence for file recovery. Waiting to try to restore the files makes it more likely that data will be overwritten, impacting actual restorability.
8. **Encrypted File Systems:** The problem with recovering ext4 is that it's often found as a part of encrypted partitions. Yet to unseal and read the information stored on encrypted Ext4 partitions needs other programs which give users different kinds of decrypting key.
9. **Data Carving Challenges:** If the metadata is damaged or missing, file carving (recovery of data through a signature) does not work. A number of fragmented and partially overwritten files make accurate data carving complex.

Personal Experience and Mitigation:

Practically speaking, when I dealt with Ext4 file systems for forensic work, they required a multi-dimensional approach. This understanding of Ext4's internals, as well as its metadata structures and allocation methods became fundamental. To overcome these challenges:

1. **In-Depth Understanding:** I went deeper into Ext4 internals, looking at metadata structures, block allocation methods and journaling operations. It helped to explain file system complexities.
2. **Real-Time Recovery Approach:** If data loss or file deletion was discovered swiftly, the likelihood of successful recovery increased. But quick response eliminated the risk that data could be overwritten.
3. **Specialized Tools and Techniques:** The use of specialized tools, such as extundelete and debugfs graphical interface software along with data carving techniques further improved the recovery process. This combined tool expertise and forensic knowledge [9].
4. **Adaptability and Continuous Learning:** In Ext4, forensic investigation required some flexibility. Enhancing recovery success rates required keeping both forensic techniques and tools, as well as file system architectures up to date.

Executing Ext4 forensics presents two primary challenges: the tools, file system architecture and data recovery techniques. To overcome these problems, one often needs to have responsiveness and experience combined with ongoing learning when it comes to digital technology.

Factors Affecting Forensic Analysis:

1. **Time Sensitivity:** One of the most important prerequisites is rapid evidence collection. But it's easy to change, remove or overwrite digital data. In such digital environments, authenticating the evidence must be rapid.
2. **Technology Evolution:** Since technology progresses, new file systems, encryption methods and storage materials are introduced. These modifications must be strictly adhered to in order for effective forensic analysis.
3. **Data Volume and Complexity:** Large-scale investigations, and especially those involving corporate culture are most aggrieved by the mass of digital data.
4. **Legal and Ethical Considerations:** First should come: respect for the law, privacy legislation and moral principles. Standard legal procedures require analysts to conserve evidence without compromising privacy.
5. **Resource Availability:** The most critical influence on determining the quality and extent of analysis is limited by resources including expertise in specialized gear and software, as well a larger number of forensic specialists.

Limitations and Challenges Faced by Analysts:

1. **Encryption and Security Measures:** Forensic investigation of encrypted data is hampered by strong security procedures, combined with safe deletion techniques. Looking at the amount of additional decryption keys or special equipment needed to deal with this wide variety evidence, one can see that there really are problems here.
2. **Data Overwriting and Fragmentation:** Contradictory and incomplete evidence the scattered nature of the data means that sectors have been overwritten by newly added information, or file system architectural confusion.
3. **Volatility and Data Alteration:** In cloud environments or volatile memory, information can be changed and deleted at any time. Nor on the other hand, can evidence be so easily obtained and preserved that its credibility remains intact.
4. **Metadata Limitations:** Loss or damage to metadata, including timestamps and file attributes can make it impossible to reconstruct the course of events or establish a chronology for actions taken on digital assets.

5. **Forensic Tool Limitations:** Some of the most important factors include whether forensic tools are effective and compatible with various file systems, encryption methods or constantly advancing technology. All this can influence both the scope and accuracy of analysis.
6. **Human Error and Bias:** In turn, analysts' interpretations, biases or omissions can affect the result of their analysis and strict procedures as well as cross-verification techniques are required to avoid errors.

This is in fact what it takes to put together and navigate through all these factors: a combination of hands-on tech skills, legal knowhow and methodological rigor so that the digital evidence will be admissible as forensic proof.

Reflective Conclusion

As we probed into Ext4 file system analysis and forensic examination, many revelations and discoveries arose. These have helped all of us to understand better the depths and intrigues within digital forensics.

Summary of Key Findings:

Upon close examination, it turned out that Ext4 might be robust but comes with its own headaches when used in forensic analysis. Mapping the contents of files to blocks and inodes revealed just how complex file storage was, followed by aspects like that data fragments interact with where they are stored. But the unpredictability of file deletions on the structure and organization of digital data also revealed just how fragile this new form is, opening up a possibility for critical information to escape or be irrevocably altered.

Insights Gained from the Analysis:

This journey through Ext4 file system analysis revealed a fine balance between data integrity and volatility. During the examination of superblocks, inodes and file system structures we saw that digital storage exists as an intricate web of interrelated elements. In addition, the difficulties encountered in recovering files revealed to an even greater extent the intrinsic limitations and complexities of getting back deleted or damaged data. This need has led many times over to both comprehensive methods as well as specialized tools developing rapidly out of these needs for such techniques.

Concluding Remarks on Ext4 File System Analysis:

It has been an interesting exploration, and also a reminder of the many dimensions to digital forensics. This analysis allowed us to view the structure of file systems, data layout and recovery from different angles. However, it also speaks to the ever-changing nature of technology-which means that traditional forensic approaches must keep developing as well. In view of the difficulties involved in probing Ext4, a multidisciplinary approach had to be adopted. Besides technological expertise, legal knowledge and methodological strictness are also required to produce complete information that includes all the angles.

Finally, Ext4 file system analysis research indicates digital forensics is both complicated and changing. It means that research and technological upgrading must be continuous, but also an overall attitude in accord with current developments. Likewise, as technology changes, so must our methodologies and conceptual frameworks. This alone can safeguard digital investigators' integrity and reliability as the environment in which they operate becomes ever more complex.

References

- [1] Ariel, "Ext2 vs Ext3 vs Ext4 File System: Which One Should You Use?," *MiniTool Partition Wizard*, 8 June 2021.
- [2] D. Both, "An introduction to Linux's EXT4 filesystem," *opensource.com*, 25 May 2017.
- [3] J. S. Katherine J. Klein, "The Challenge of Implementation," *Academy of Management Review*, vol. 21(4), pp. 1055-1080, October 1996.
- [4] J. Salter, "Understanding Linux filesystems: ext4 and beyond," *opensource.com*, 2 April 2018.
- [5] H. Pomeranz, "Understanding EXT4 (Part 1): Extents," *SANS*, 20 December 2010.
- [6] T. Fisher, "What Is a Root Folder or Root Directory?," *Life Wire: Tech for Humans*, 9 September 2021.
- [7] E. Simard, "Everything You Need to Know About inodes in Linux," *Linux Handbook*, 7 March 2019.
- [8] E. BUCHANAN, "How To Undelete Files on Ext4 Partition," *Devicetests*, 21 JULY 2023.
- [9] K. D. Fairbanks, "An analysis of Ext4 for digital forensics," *Digital Investigation*, vol. 9, pp. 118-S130, August 2012.
- [10] M. O. Y. U. S. K. T. N. S. T. K. Yuko Sakai-Tagawa, "Detection sensitivity of influenza rapid diagnostic tests," *Microbiol Immunol.*, vol. 58(10), p. 600–606., 1 October 2014 .